

VS2022调试工具

使用报告



班级：计拔/高程05班

姓名：毛星博

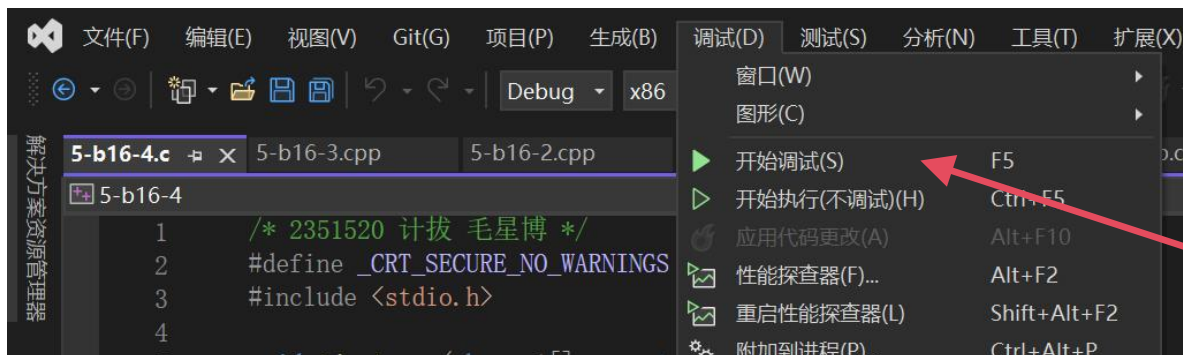
学号：2351520

完成日期：2024年5月26日

1. VS2022 下调试工具的基本使用方法

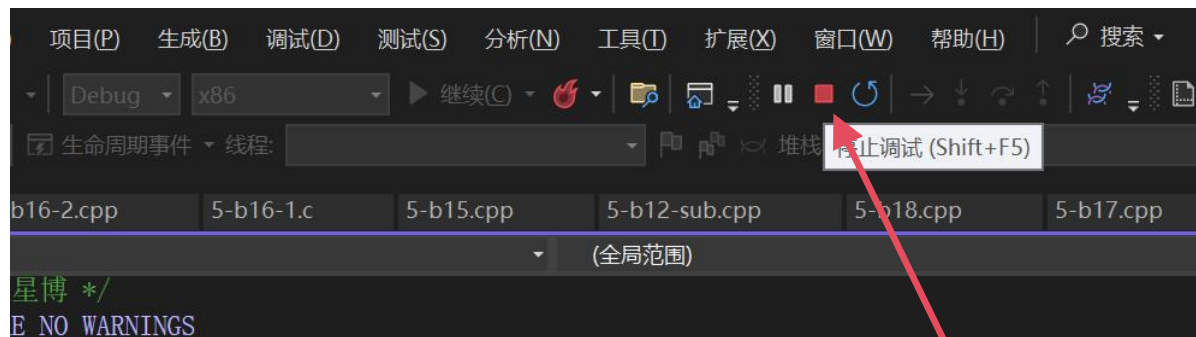
1.1 如何开始调试？如何结束调试？

1.1.1 如何开始调试



如图所示，在“调试”窗口点击“开始调试”，或直接按快捷键F5，或在主界面按“本地Windows调试器”开始调试。

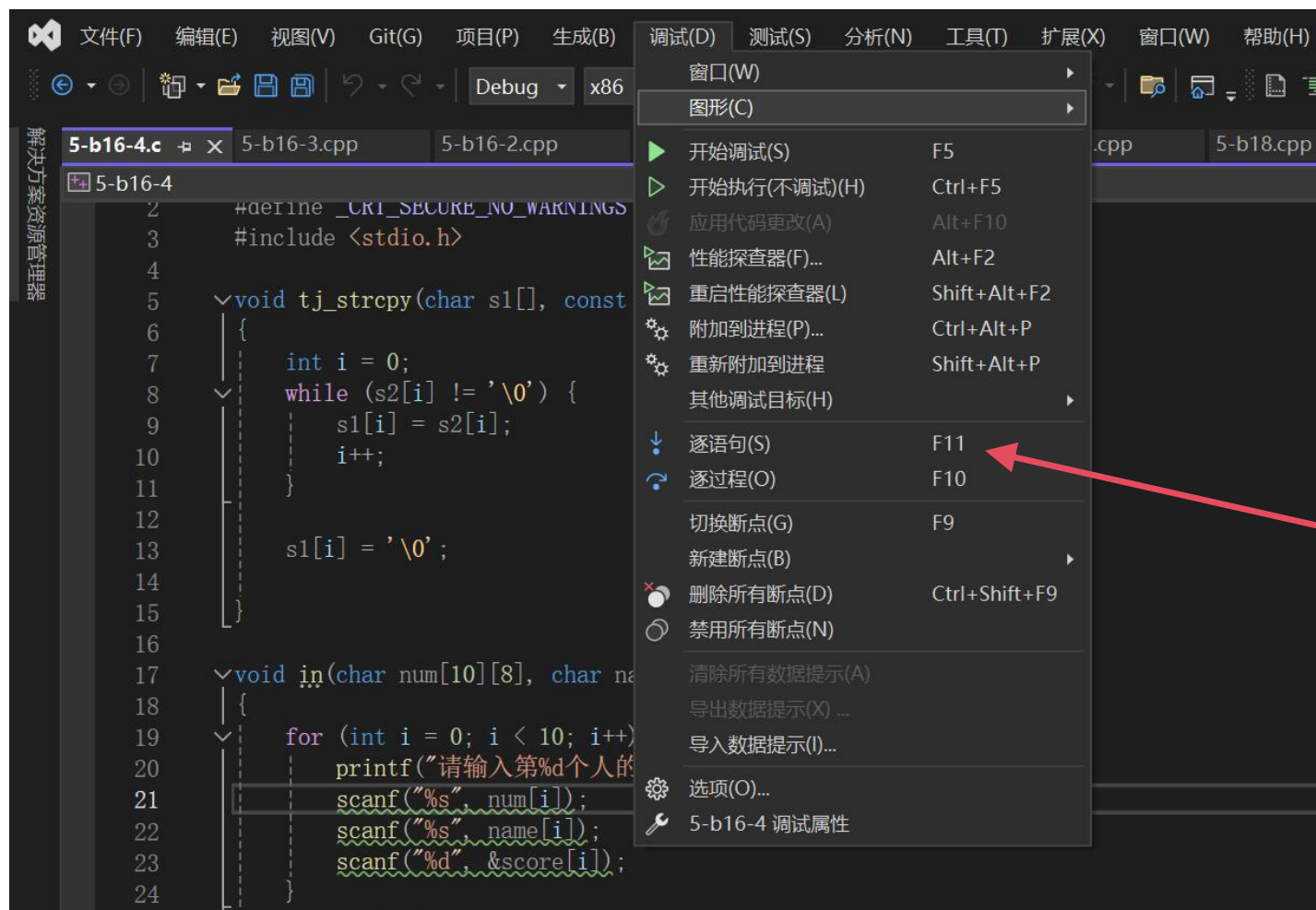
1.1.2 如何结束调试



如图所示，按下“停止调试”或直接按快捷键Shift+F5键即可结束调试。

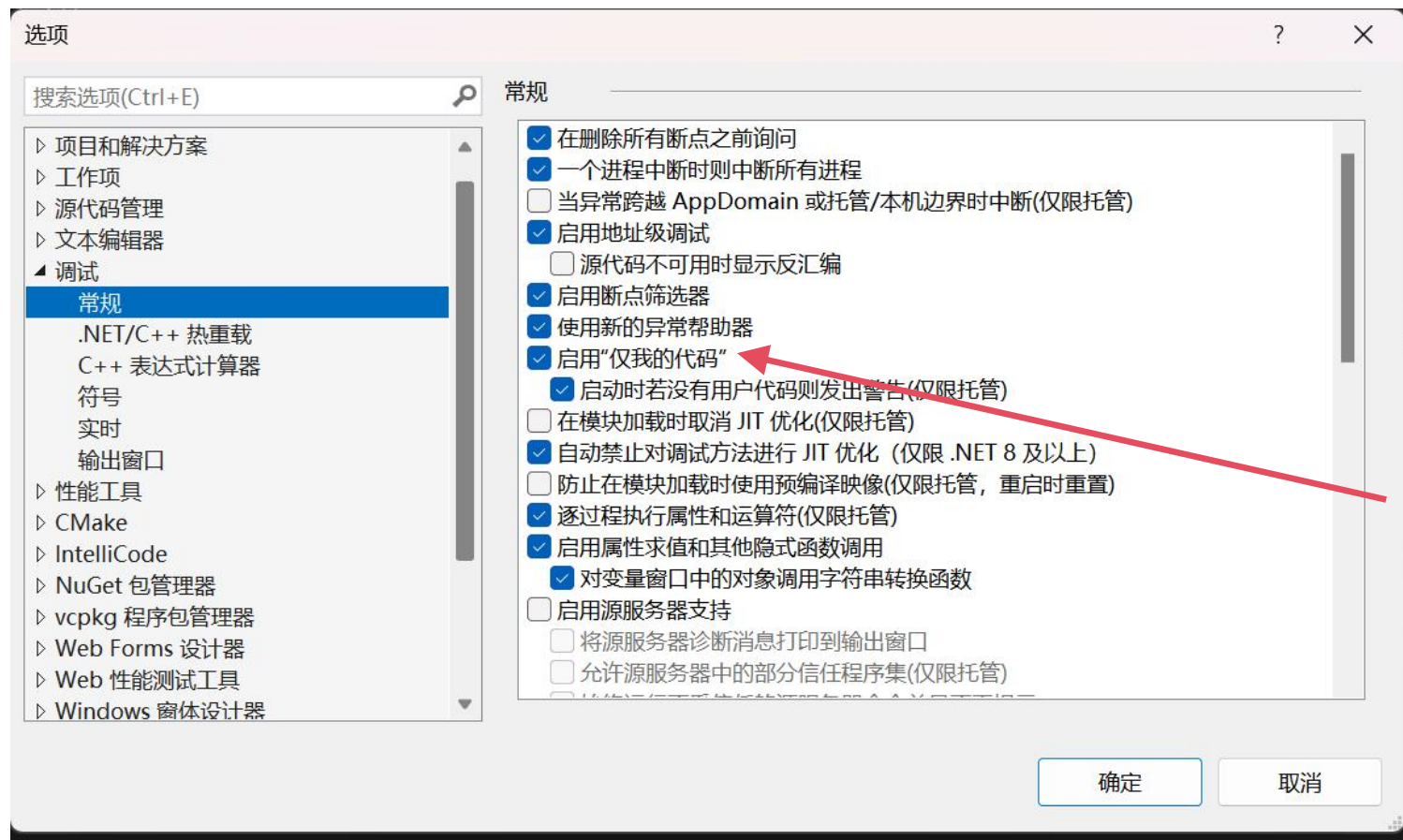
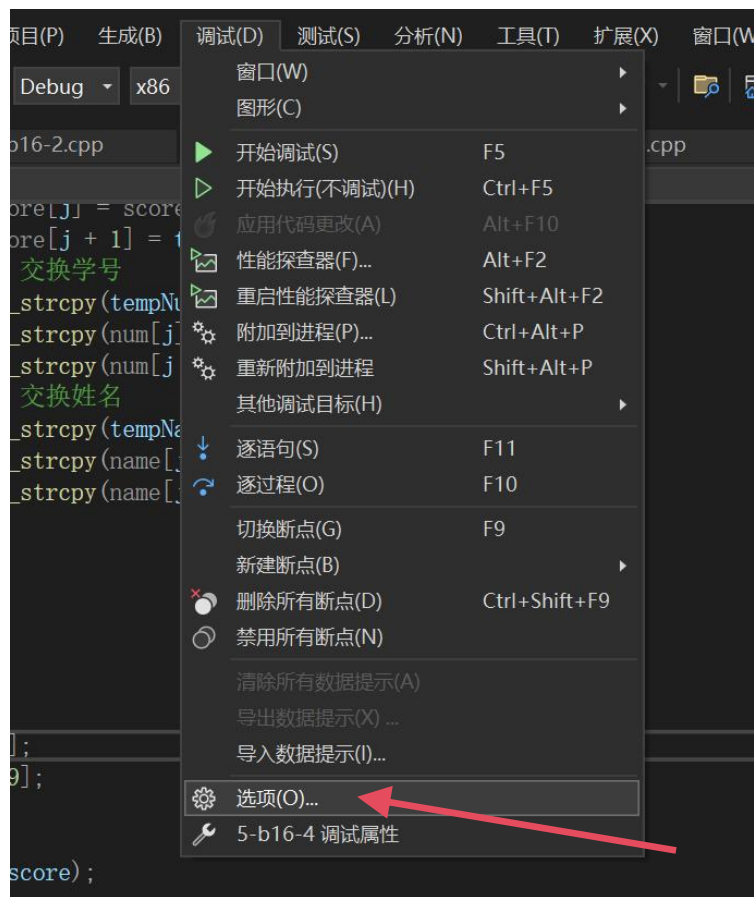
注：如果不设置断点直接开始调试，程序会在调试完成后自动结束，如果设置了断点或者选择逐语句/逐过程调试，则需要手动结束或者完成所有语句后结束调试。

1.2 如何在一个函数中每个语句单步执行？



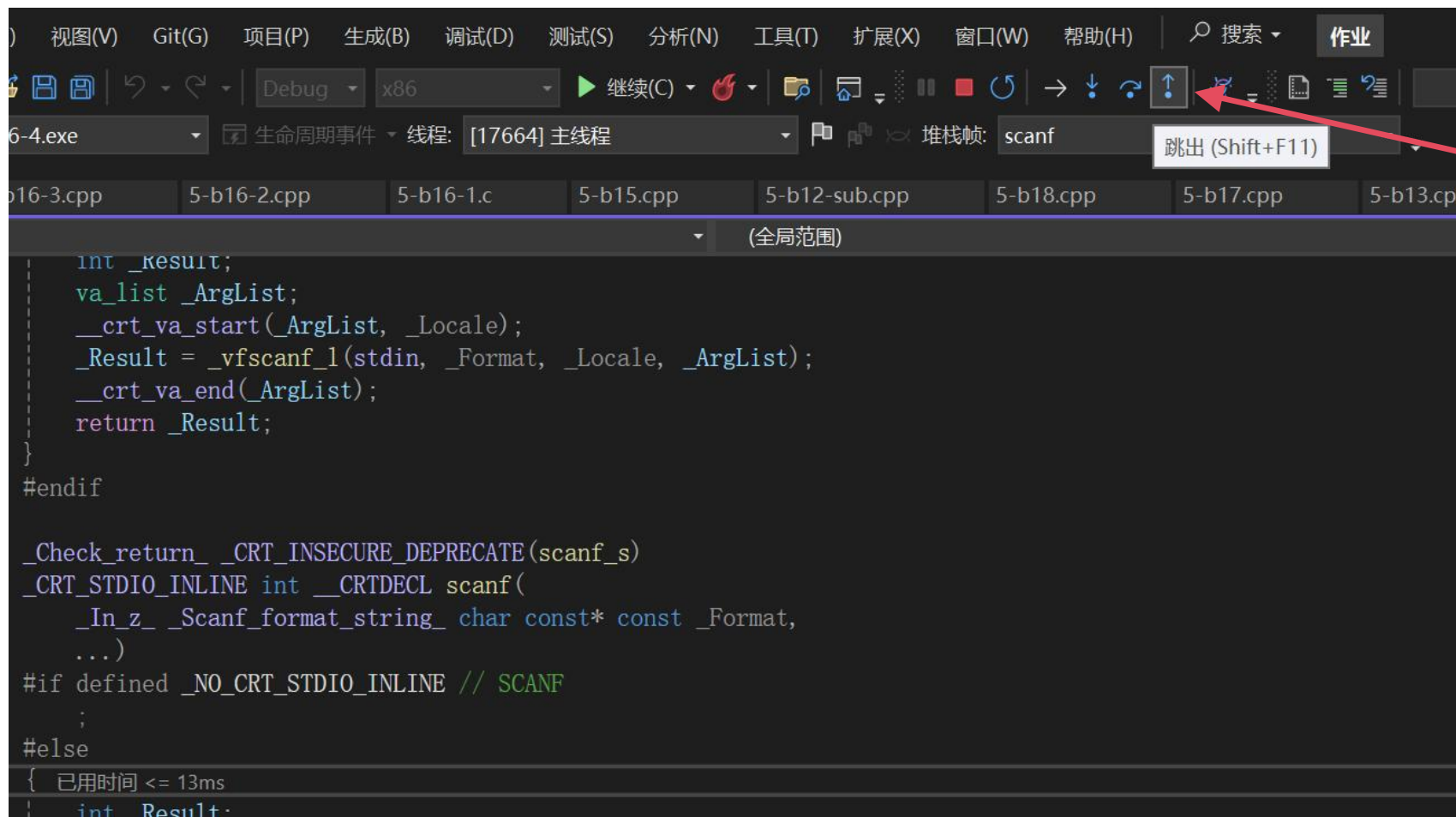
如图所示，在调试窗口点击“逐语句”，或者按快捷键F11，即可实现每个语句单步执行。

1.3 在碰到 cout/sqrt 等系统类/系统函数时，如何一步完成这些系统类/系统函数的执行而不要进入到这些系统类/函数的内部单步执行？



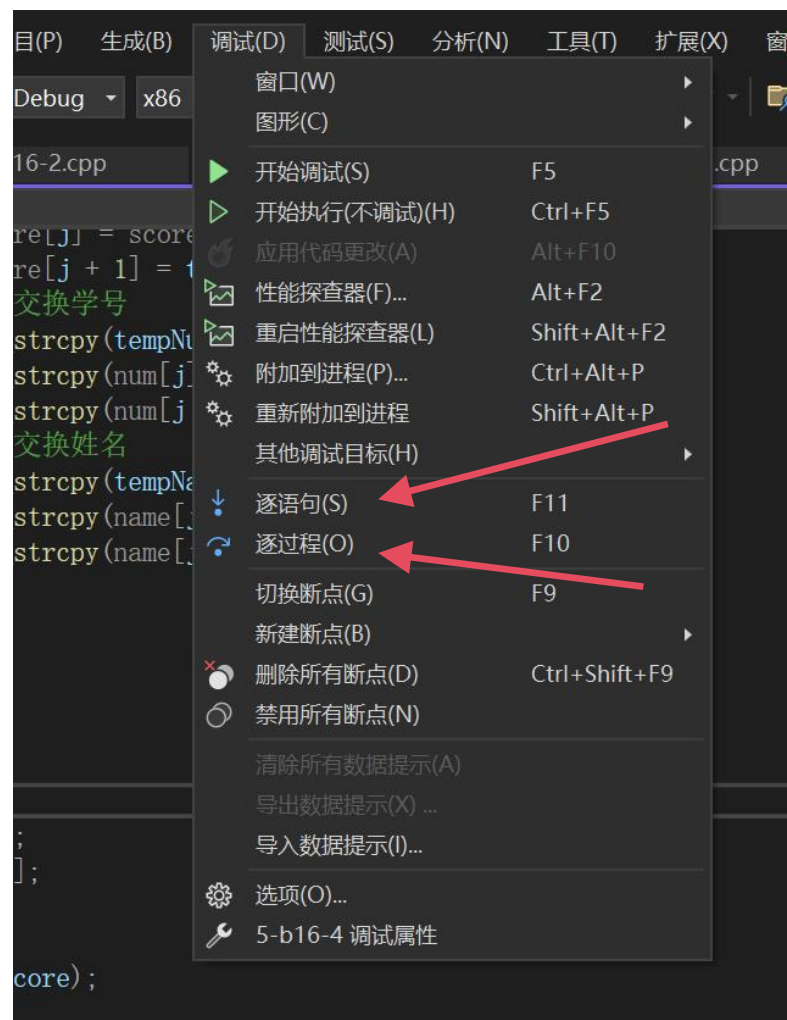
如图所示，在调试窗口点击“选项”，进入选项界面后，勾选“启用仅我的代码”，此时在调试时碰到 cout/sqrt 等系统类/系统函数时，就会一步完成这些系统类/系统函数的执行，而不会进入到这些系统类/函数的内部单步执行。（无论是逐过程还是逐语句）

1.4 如果已经进入到 cout/sqrt 等系统类/系统函数的内部，如何跳出并返回自己的函数？



如图所示，已经进入到 cout/sqrt 等系统类/系统函数的内部时，只需点击界面上方的“跳出”，或者按快捷键Shift+F11，即可跳出并返回自己的函数。

- 1.5 在碰到自定义函数的调用语句时，如何一步完成自定义函数的执行而不进入到这些自定义函数的内部单步执行？
- 1.6 在碰到自定义函数的调用语句时，如何转到被调用函数中单步执行？



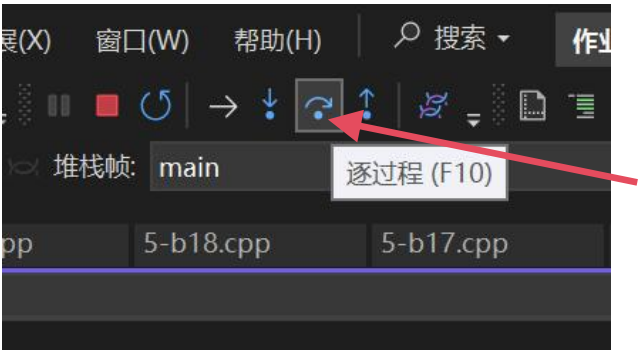
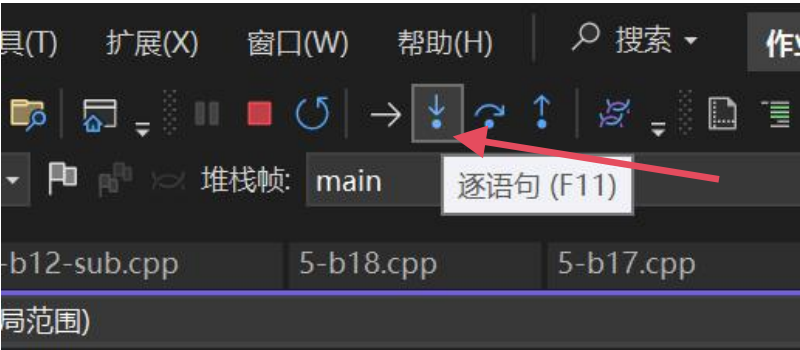
```
int main()
{
    char num[10][8];
    char name[10][9];
    int score[10];

    in(num, name, score); 已用时间 <= 21ms
    //printf("%s %s %d", num[0], name[0], score[0]); (测试)

    paixu(num, name, score);

    out(num, name, score);

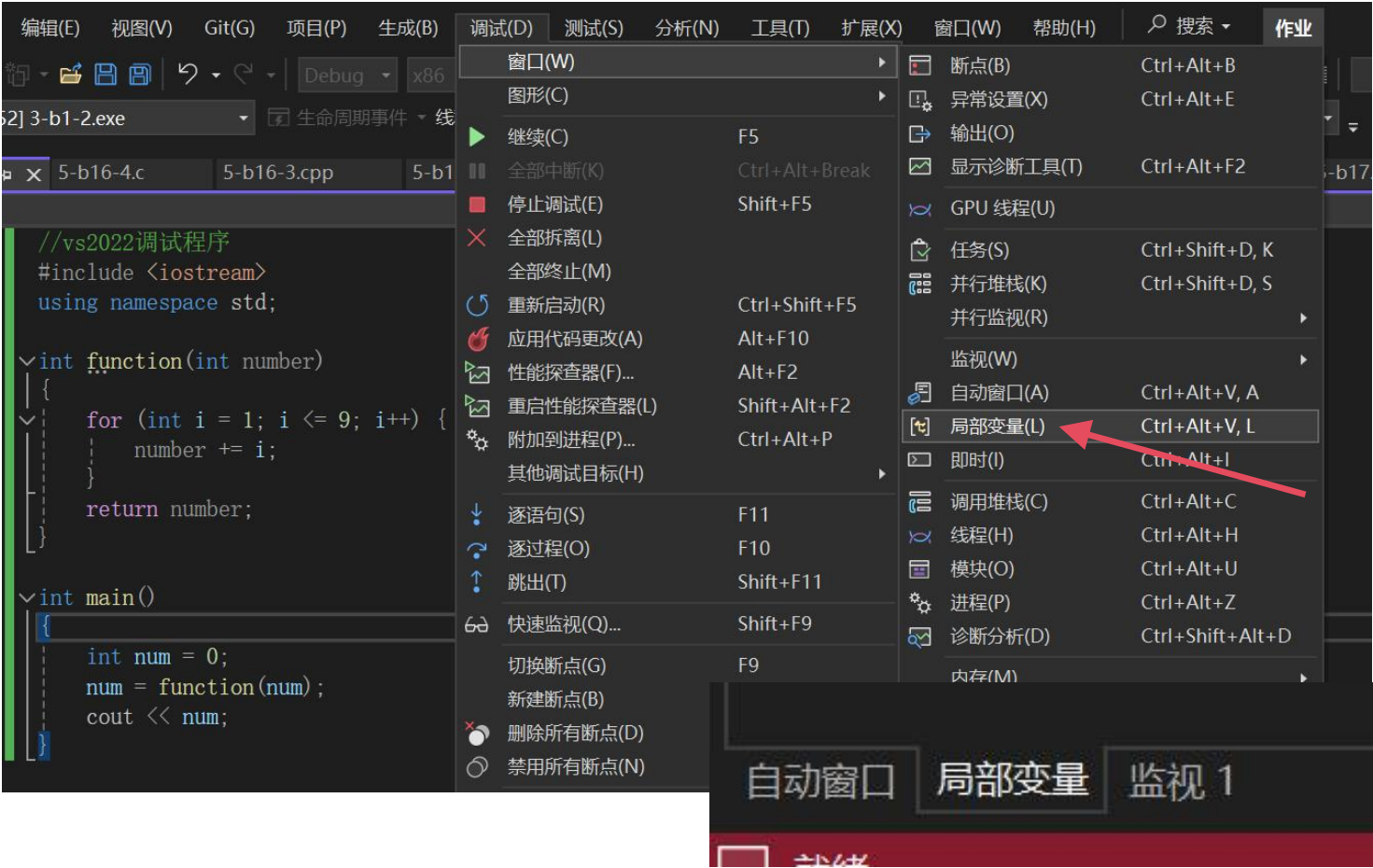
    return 0;
}
```



如图所示，按下“逐语句”或“逐过程”开始调试后，例如此时图上调试光标在in函数处，此时点击“逐语句”或按F11，即可转到被调用函数中单步执行；点击“逐过程”或按F10，即可一步完成自定义函数的执行而不进入到这些自定义函数的内部单步执行。

2.掌握用 VS2022 的调试工具查看各种生存期/作用域变量的方法

2.1 查看形参/自动变量的变化情况 (本页涉及知识点4.7.1)



名称	值
i	1
number	0

名称	值
i	5
number	15

名称	值
i	9
number	45

先选择“逐语句”或“逐过程”或设置断点，进入可控调试阶段，然后依次点击“调试”-“窗口”-“局部变量”或按快捷键Ctrl+Shift+V或直接选择局部变量窗口，再依次执行语句或过程，即可查看形参和自动变量的变化情况。

2.2 查看静态局部变量的变化情况（该静态局部变量所在的函数体内/函数体外） **(本页涉及知识点4.7.1/4.8.2/4.9.3)**

```
//vs2022调试程序
#include <iostream>
using namespace std;

int function(int number)
{
    for (int i = 1; i <= 9; i++) {
        static int a = 1;
        number += a; 已用时间 <= 9ms
        a++;
    }
    return number;
}

int main()
{
    int num = 0;
    num = function(num);
    cout << num;
}
```

名称	值
 a	1
 i	1
 number	0

```
//vs2022调试程序
#include <iostream>
using namespace std;

int function(int number)
{
    for (int i = 1; i <= 9; i++) {
        static int a = 1;
        number += a; 已用时间 <= 8ms
        a++;
    }
    return number;
}

int main()
{
    int num = 0;
    num = function(num);
    cout << num;
}
```

名称	值
 a	2
 i	2
 number	1

```
//vs2022调试程序
#include <iostream>
using namespace std;

int function(int number)
{
    for (int i = 1; i <= 9; i++) {
        static int a = 1;
        number += a;
        a++;
    }
    return number;
}

int main()
{
    int num = 0;
    num = function(num);
    cout << num; 已用时间 <= 9ms
}
```

名称	值
 已返回 function	45
 num	45

观察静态局部变量方法与2.1相同

如图所示，本例中的静态局部变量a的定义语句在调试时被直接跳过；再一次执行定义语句时，a保留上次的值而不会被重新赋值。

可以看到，虽然静态局部变量的生存期为整个程序执行过程，但是只有调试过程执行到其作用域(该静态局部变量所在的函数体内)时，静态局部变量才会显示在自动窗口中，在函数体外不会显示。(因为在作用域之外，静态局部变量无法被访问)

2.3 查看静态全局变量的变化情况（两个源程序文件，有静态全局变量同名） （本页涉及知识点4.7.2/4.8.5）

```
调试.cpp  调试-主.cpp  5-b16-4.c  5-b16-3.cp
3-b1-2
1  //vs2022调试程序
2  #include <iostream>
3  using namespace std;
4
5  static int a = 1;
6
7  void fun();
8
9  int function(int number)
10 {
11     for (int i = 1; i <= 9; i++) {
12         number += a; 已用时间 <= 5ms
13         a++;
14     }
15     return number;
16 }
17
18 int main()
19 {
20     fun();
21     int num = 0;
22     num = function(num);
23     cout << num;
24 }
```

名称	值
 a	1
 i	1
 number	0

如图所示，可以看到在同一个项目中的两个源程序文件中分别定义了变量名为a的静态局部变量

```
调试.cpp  调试-主.cpp  5-b16-4.c
3-b1-2
1  #include <iostream>
2  using namespace std;
3
4  static int a = 100;
5
6  void fun()
7  {
8      a++;
9      已用时间 <= 6ms
```

名称	值
 a	101

在调试过程中，在自动窗口中，一个文件的静态全局变量只有在执行到该文件的语句中时才会显示，两个同名静态全局变量互不影响。

2.4 查看外部全局变量的变化情况（两个源程序文件，一个定义，另一个有 extern 说明） (本页涉及知识点4.7.2/4.8.4)

```
//vs2022调试程序
#include <iostream>
using namespace std;

int a = 1;
void fun();

int function(int number)
{
    for (int i = 1; i <= 9; i++) {
        number += a; 已用时间 <= 6ms
        a++;
    }
    return number;
}

int main()
{
    fun();
    int num = 0;
    num = function(num);
    cout << num;
}
```

名称	值
 a	2
 i	1
 number	0

如图所示，可以看到在一个源程序中定义了外部全局变量a，另一个源程序中用extern声明了变量a。

```
调试.cpp  X  调试-主.cpp  5-b16-4.c
3-b1-2
1  #include <iostream>
2  using namespace std;
3
4  extern int a;
5
6  void fun()
7  { 已用时间 <= 7ms
8      a++;
9  }
```

名称	值
 a	1

名称	值
 a	2

此时两个源程序文件中使用的是同一个变量a，a每次使用时保留上次的值。

2.5 补充：当只想观察某个或某些特定的变量变化情况时，可以使用监视窗口，在监视窗口中添加想要观察的变量，即可有目标的观察特定的变量变化情况。

```
调试.cpp  调试-主.cpp  5-b16-4.c  5-b16-3.cpp  5-  
3-b1-2  
1  //vs2022调试程序  
2  #include <iostream>  
3  using namespace std;  
4  
5  int a = 1;  
6  
7  void fun();  
8  
9  int function(int number)  
10 | {  
11 |     for (int i = 1; i <= 9; i++) {  
12 |         number += a;  
13 |         a++;  
14 |     }  
15 |     return number;  
16 | }  
17  
18 int main()  
19 | {  
20 |     fun();  
21 |     int num = 0;  
22 |     num = function(num); 已用时间 <= 1ms  
23 |     cout << num;  
24 | }
```

名称	值
 a	2
 num	0
添加要监视的项	
自动窗口 局部变量 监视 1	

3.掌握用 VS2022 的调试工具查看各种不同类型变量的方法

- 3.1 char/int/float 等简单变量 (本题涉及知识点2.5)
- 3.2 指向简单变量的指针变量 (如何查看地址、值?) (本题涉及知识点6.2)
- 3.3 一维数组 (本题涉及知识点5.2)
- 3.4 指向一维数组的指针变量 (如何查看地址、值?) (本题涉及知识点6.3)

```
调试.cpp | 调试-主.cpp* | 5-b15.cpp | 5-b12-s
+ 3-b1-2
1 //vs2022调试程序
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     char a = 'a';
8     int b = 6;
9     float c = 6.6f;
10    int d[6] = { 1, 2, 3, 4, 5, 6 };
11
12    char* da = &a;
13    int* db = &b;
14    float* dc = &c;
15    int* dd1 = d;
16    int* dd2 = &d[1];
17 }
```

名称	值	类型
a	97 'a'	char
b	6	int
c	6.59999990	float
d	0x0086fa48 {1, 2, 3, 4, 5, 6}	int[6]
da	0x0086fa83 <字符串中的字符无效。>	char *
db	0x0086fa74 {6}	int *
dc	0x0086fa68 {6.59999990}	float *
dd1	0x0086fa48 {1}	int *
dd2	0x0086fa4c {2}	int *

如图所示，char/int/float 等简单变量、一维数组以及指向简单变量和一维数组的指针变量的值或指向的地址(简单变量不会显示地址，数组和指针变量会显示地址)都可以在变量的窗口中查看，查看方法与《2.掌握用 VS2022 的调试工具查看各种生存期/作用域变量的方法》相同。

另外，指向简单变量的指针变量会同时显示指向的地址和其中存的值；而指向一维数组的指针变量则会显示指向的地址和指向值在数组中的位置，如本例中，指针变量dd1指向数组d的第一个变量，则显示{1}，指针变量dd2指向数组d的第二个变量，则显示{2}。

注：一维字符数组在内部存储方式为字符串，这在3.5中会详细说明。

3.5 二维数组（包括数组名仅带一个下标的情况） **（本题涉及知识点5.3）**

```
调试-主.cpp 5-b15.cpp 5-b12-sub.cpp 5-b18.cpp

//vs2022调试程序
#include <iostream>
using namespace std;

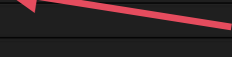
int main()
{
    int a[2][2] = { 1, 2, 3, 4 };
    int b[][3] = { 5, 6, 7, 8, 9, 0 };
    char c[2][4] = { {'a', 'b'}, {'c', 'd', 'e'} };

    int* da = a[0];
    int* db = &b[1][1];

    char* dc1 = c[0];
    char* dc2 = &c[0][1];

    cout << a[1];
    cout << b[0][2];
}
已用时间 <= 31ms
```

注：在定义指向二维字符数组的指针时，不能直接用二维字符数组名作为地址，这是因为二维数组在系统中的存储方式为“一维数组的一维数组”，二维字符数组名代表指向数组首行的指针(指向数组的指针)，这与我们一般使用的指向数值的指针类型不匹配。

名称	值	类型
a	0x0093facc {0x0093facc {1, 2}, 0x0093fad4 {3, 4}}	int[2][2]
b	0x0093faac {0x0093faac {5, 6, 7}, 0x0093fab8 {8, 9, 0}}	int[2][3]
c	0x0093fa9c {0x0093fa9c "ab", 0x0093faa0 "cde"}	char[2][4]
da	0x0093facc {1}	int *
db	0x0093fab8 {9}	int *
dc1	0x0093fa9c "ab" 	char *
dc2	0x0093fa9d "b" 	char *

名称	值	类型
a	0x0053f854 {0x0053f854 {1, 2}, 0x0053f85c {3, 4}}	int[2][2]
a[1]	0x0053f85c {3, 4}	int[2]
b	0x0053f834 {0x0053f834 {5, 6, 7}, 0x0053f840 {8, 9, 0}}	int[2][3]
b[0]	0x0053f834 {5, 6, 7}	int[3]
b[0][2]	7	int

如图所示，二维数组在显示时，会显示数组首地址、每行首地址以及每行各元素的值；这应该与二维数组在系统中的存储方式为“一维数组的一维数组”有关。

另外，可以看到二维字符数组在系统内部的存储方式为每行以字符串方式存储（可以验证一维字符数组也是如此）

当数组名仅带一个下标时，在“值”一栏中会显示其指向该行的首元素的地址，并显示该行所有元素的值。

另外，当输出二维数组中某一个元素或某一行的值时，自动窗口会显示每一级的数据，例如，当本例中输出b[0][2]时，会依次显示b、b[0]、地址和值，b[0][2]的值（单个元素不显示地址）。（还挺人性化）

3.6 实参是一维数组名，形参是指针的情况，如何在函数中查看实参数组的地址、值？ (本题涉及知识点6.3.5)

```
//vs2022调试程序
#include <iostream>
using namespace std;

void fun(int* p1, char* p2)
{
    p1[2] = 6;
    p2[2] = 'x';
}

int main()
{
    int a[3] = { 1,2,3 };
    char b[3] = { 'a','b','c' };
    fun(a, b);
    cout << a[2] << endl;
    cout << b[2] << endl;
    return 0;
}
```

堆栈帧: main

3-b1-2.exe!fun(int * p1, char * p2) 行 9

3-b1-2.exe!main() 行 15

[外部代码]

[下面的框架可能不正确和/或缺失, 没有为 kernel32.dll 加载符号]

如图所示，在堆栈帧中选择main函数，即可查看实参数组的地址和值。

```
//vs2022调试程序
#include <iostream>
using namespace std;

void fun(int* p1, char* p2)
{
    p1[2] = 6;
    p2[2] = 'x';
}

int main()
{
    int a[3] = { 1,2,3 };
    char b[3] = { 'a','b','c' };
    fun(a, b);
    cout << a[2] << endl;
    cout << b[2] << endl;
    return 0;
}
```

名称	值
p1	0x012ffd28 {1}
p1[2]	6
p2	0x012ffd1c <字符串中的字符无效。>
p2[2]	120 'x'

名称	值
a	0x006ffbd4 {1, 2, 6}
b	0x006ffbc8 "abx..."

如图，此时调试到fun函数中时，只能看到形参指针指向的地址(实际上就是实参数组的地址)，但不能看到实参数组的值

3.7 指向字符串常量的指针变量（能否看到无名字符串常量的地址？）（本题涉及知识点6.4）

```
//vs2022调试程序
#include <iostream>
using namespace std;

int main()
{
    const char* p = "abcdefg";
    cout << p << endl;
    cout << *p << endl;
    p++; 已用时间 <= 38ms
    cout << p << endl;
    cout << *p << endl;
}
```

名称	值
*p	97 'a'
p	0x005c9bcc "abcdefg"

```
//vs2022调试程序
#include <iostream>
using namespace std;

int main()
{
    const char* p = "abcdefg";
    cout << p << endl;
    cout << *p << endl;
    p++;
    cout << p << endl;
    cout << *p << endl;
    已用时间 <= 41ms
}
```

名称	值
*p	98 'b'
p	0x005c9bcd "bcdefg"

如图所示，字符指针常变量p指向无名字符串常量“abcdefg”，可以看到字符串常量的地址。

定义一个字符指针常变量p，p的值是可以改变的(如图)，但p指向的地址中存的值不可改变。例如，如果写语句：(*p)++则会报错。

3.8 引用（引用与指针是否有区别？有什么区别？）（本题涉及知识点6.7）

引用和指针当然有区别。

- 1、从含义上说，引用是变量的别名，不分配单独的空间；而指针则是存放一个变量的地址的变量，有单独分配的空间。
- 2、引用必须在声明时进行初始化，指向同类型的变量，且不能更改其指向的对象；而指针可以在定义时不初始化，并且可以随时改变其指向的对象。
- 3、作为函数的形参时，指针的本质还是单向传值，只是传递的是变量的地址，间接访问和修改变量，而引用则直接是将实参变量的地址传给形参，直接对实参操作。
- 4、引用必须指向一个具体的变量，不能空引用；而指针可以允许空指针，也就是指向NULL。

3.8.1 指针作为形参 (本例涉及知识点6.2.5)

```
//vs2022调试程序
#include <iostream>
using namespace std;

void fun(int* p, int* q)
{
    int t;
    t = *p;
    *p = *q;
    *q = t;
}

int main()
{
    int i = 6, j = 66;
    fun(&i, &j);
    cout << i << j;
}
```

名称	值
▷ &i	0x005ffb88 {6}
▷ &j	0x005ffb7c {66}
i	6
j	66

名称	值
▷ p	0x007cfc98 {6}
▷ q	0x007cfc8c {66}

名称	值
*p	66
*q	6
t	6

可以看到，指针p和q存放的是i和j的值，通过访问i和j的地址来交换两个变量的值

3.8.2 引用作为形参 (本例涉及知识点6.7.2)

```
//vs2022调试程序
#include <iostream>
using namespace std;

void fun(int& p, int& q)
{
    int t;
    t = p;
    p = q;
    q = t;
}

int main()
{
    int i = 6, j = 66;
    fun(i, j);
    cout << i << j;
}
```

名称	值
 p	6
 q	66

名称	值
 p	66
 q	6
 t	6



名称	值
 i	66
 j	6



可以看到，引用p、q作为函数的形参时，直接改变其指向的变量i和j的值

3.9 使用指针时出现了越界访问 (本题涉及知识点6.3)

```
//vs2022调试程序
#include <iostream>
using namespace std;

int main()
{
    int a[3] = { 1, 2, 3 };
    int* p = a;
    p += 3;
    cout << *p;
    *p = 4;
}
```

名称	值
*p	-858993460
a	0x010ff890 {1, 2, 3}
p	0x010ff89c {-858993460}

```
//vs2022调试程序
#include <iostream>
using namespace std;

int main()
{
    int a[3] = { 1, 2, 3 };
    int* p = a;
    p += 3;
    cout << *p;
    *p = 4;
}
```

已引发异常

Run-Time Check Failure #2 - Stack around the variable 'a' was corrupted.

[显示调用堆栈](#) | [复制详细信息](#)

名称	值
*p	4
p	0x010ff89c {4}

可以看到，此时指针p已经越界，此时输出p指向的地址中存的值为随机值，属于非法读（不会报错）；而后尝试修改p指向地址中的值，属于非法写，此时出现异常。